

**UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA**



**ANÁLISIS E IMPLEMENTACIÓN DE ALGORITMOS
PARA ÁRBOLES DE EXPANSIÓN MÍNIMA
(KRUSKAL Y PRIM)**

Grupo 7

Integrantes:

Ibarra Sanchez, Jean Paul Fernando	20261584K
Valencia Casanova, Renzo Leiser	20231267G
Romero Cerron, Byron Samir	20262162B

Docente: M.Sc. Luis Antonio Selis Vasquez

Curso: Algoritmos y Estructura de Datos I

Código: BMA09Q

Fecha de entrega: 05/10/2026

ÍNDICE

1. INTRODUCCIÓN	3
2. ÁRBOL DE EXPANSIÓN	4
2.1. Características en un grafo conexo.....	4
3. ÁRBOL DE EXPANSIÓN MÍNIMA (MST)	4
3.1. Características.....	5
3.2. Algoritmos estudiados (ambos voraces).....	5
3.3. Implementación en C.....	5
3.4. Complejidad computacional.....	6
3.5. Aplicaciones	6
4. ALGORITMO DE KRUSKAL	6
4.1. Funcionamiento	6
4.2. Detección y evitación de ciclos	7
4.3. Análisis de complejidad computacional.....	8
4.3.1. Qué representa la complejidad.....	8
4.3.2. Complejidad temporal: $O(m \log n)$	8
4.3.3. Complejidad espacial: $O(n + m)$	9
5. ALGORITMO DE PRIM.....	9
5.1. Funcionamiento.....	9
5.2. Análisis de complejidad computacional.....	10
5.2.1. Qué representa la complejidad.....	10
5.2.2. Complejidad temporal.....	10
5.2.3. Complejidad espacial.....	11
6. COMPARACIÓN ENTRE KRUSKAL Y PRIM	12
6.1. Tabla comparativa entre los algoritmos de Kruskal y Prim.....	12
6.2. Consideraciones para su implementación en Lenguaje C.....	13
7. SELECCIÓN DEL ALGORITMO	13
8. EVIDENCIAS DE EJECUCIÓN	14
9. CONCLUSIONES.....	15
10. BIBLIOGRAFÍA	16

1. INTRODUCCIÓN

Los grafos constituyen una herramienta fundamental para representar problemas en los que intervienen conjuntos de elementos y las conexiones existentes entre ellos. Dentro de este campo, los árboles de expansión permiten conectar todos los vértices de un grafo conexo sin formar ciclos, mientras que los árboles de expansión mínima buscan, además, minimizar la suma total de los pesos asociados a sus aristas. Estas estructuras resultan especialmente útiles en problemas de optimización relacionados con redes, conexiones y distribución de recursos.

En el presente informe se estudian los principales conceptos relacionados con los árboles de expansión mínima y se analizan los algoritmos de Kruskal y Prim, ambos basados en estrategias voraces. Se describe su funcionamiento, las estructuras de datos empleadas, sus métodos de selección de aristas y las técnicas utilizadas para evitar ciclos. Asimismo, se desarrolla un análisis de su complejidad temporal y espacial considerando distintas formas de implementación.

Además del estudio teórico, ambos algoritmos son implementados en lenguaje C y se comparan considerando criterios como eficiencia, densidad del grafo, representación de los datos, facilidad de implementación y comportamiento frente a grafos desconectados. Finalmente, se presentan evidencias de ejecución que permiten comprobar el funcionamiento de las implementaciones y verificar la obtención del árbol de expansión mínima.

2. ÁRBOL DE EXPANSIÓN

Un árbol de expansión (también llamado *árbol recubridor* o *spanning tree*) de un grafo conexo y no dirigido $G = (V, E)$ es un subgrafo $T = (V, E')$ que contiene todos los vértices de G y es un árbol (un grafo conexo sin ciclos).

En términos sencillos, es una estructura que conecta todos los vértices del grafo original sin formar ciclos y contiene exactamente $|V| - 1$ aristas.

2.1. Características en un grafo conexo

- Todo grafo conexo tiene al menos un árbol de expansión.
- Tiene exactamente $|V| - 1$ aristas.
- Entre cualquier par de vértices existe un único camino.
- Quitar una arista lo desconecta. Añadir una arista crea exactamente un ciclo.
- Un grafo puede tener muchos árboles de expansión. El grafo completo K_n tiene n^{n-2} (fórmula de Cayley). En general se cuentan con el teorema de Kirchhoff.
- Si el grafo no es conexo, solo existe un bosque de expansión.

3. ÁRBOL DE EXPANSIÓN MÍNIMA (MST)

Un árbol de expansión mínima (o de peso mínimo) es un subconjunto de aristas de un grafo que conecta todos los vértices sin formar ciclos y minimizando la suma total de sus pesos o distancias

El Problema del Árbol de Expansión Mínima consiste en encontrar, dentro de un grafo conexo, no dirigido y ponderado (donde cada arista tiene asignado un valor o peso), un árbol de expansión cuya suma total de pesos de las aristas sea la menor posible.

Formalmente: Dado un grafo conexo, no dirigido y ponderado con $w: E \rightarrow \mathbb{R}$, el problema consiste en encontrar un árbol de expansión T que minimice el costo total:

$$w(T) = \sum w(e), \text{ para } e \in T$$

3.1. Características

- Tiene $|V| - 1$ aristas.
- Si todos los pesos son distintos, el MST es único. Con pesos repetidos puede haber varios MST del mismo costo.
- Propiedad del corte: para cualquier partición de V en dos conjuntos, la arista más liviana que los cruza pertenece a algún MST.
- Propiedad del ciclo: la arista más pesada de un ciclo (si es la única máxima) no pertenece a ningún MST.
- Minimiza el costo total, pero no los caminos mínimos entre pares de vértices (eso lo resuelve Dijkstra).

3.2. Algoritmos estudiados (ambos voraces)

Kruskal:

1. Ordenar las aristas por peso ascendente.
2. Recorrerlas y añadir cada una si no forma ciclo (se verifica con Union-Find).
3. Detenerse al tener $|V| - 1$ aristas.

Prim:

1. Partir de un vértice cualquiera.
2. Añadir siempre la arista más barata que conecte el árbol con un vértice externo.
3. Terminar cuando todos los vértices estén en el árbol.

3.3. Implementación en C

- **Programas:** kruskal.c y prim.c.
- **Estructuras:** arreglo de aristas y Union-Find con compresión de caminos y unión por rango (Kruskal). Matriz o lista de adyacencia con arreglos `key[ ]` y `parent[ ]`, o heap (Prim).
- **Salida:** aristas del MST y costo total.
- El código fuente no se incluye en el informe. Se incluyen capturas de la ejecución de ambos programas utilizando el mismo grafo, con el fin de comprobar que obtienen un árbol de expansión mínima con el mismo costo total.

3.4. Complejidad computacional

Algoritmo	Tiempo	Espacio
Kruskal	$O(E \log V)$	$O(V + E)$
Prim (matriz + arreglo)	$O(V^2)$	$O(V^2)$
Prim (lista + heap binario)	$O(E \log V)$	$O(V + E)$

Qué representa: la tasa de crecimiento del tiempo y la memoria en función del tamaño del grafo (V vértices, E aristas), en el peor caso.

3.5. Aplicaciones

- Diseño de redes de costo mínimo (fibra óptica, eléctricas, agua, carreteras).
- Agrupamiento (clustering) y análisis de datos.
- Base de algoritmos de aproximación para el problema del viajante métrico (TSP métrico).
- Segmentación de imágenes.

4. ALGORITMO DE KRUSKAL

El algoritmo de Kruskal es un método voraz (*greedy*) que construye un Árbol de Expansión Mínima (MST) de un grafo $G = (V, E)$ conexo, no dirigido y con un peso $w(e)$ en cada arista. En toda la sección, $n = |V|$ es el número de vértices y $m = |E|$ el número de aristas. La idea es recorrer las aristas de menor a mayor peso y aceptar cada una siempre que no forme un ciclo, hasta conectar todos los vértices.

4.1. Funcionamiento

1. **Inicialización y ordenamiento.** Se parte de un bosque de n árboles, donde cada vértice es una componente conexa aislada. Luego se ordenan las aristas de E por peso no decreciente.
2. **Evaluación voraz.** Se examina cada arista $e = (u, v)$ en ese orden:
 - **Aceptar:** si u y v están en componentes distintas, e es la arista más barata aún no examinada que las une. Por la propiedad de corte del MST es una *arista segura*, así que se agrega al MST y ambas componentes se fusionan.

- **Descartar:** si u y v ya están en la misma componente, existe un camino entre ellos formado por aristas de peso menor o igual a $w(e)$. Agregar e cerraría un ciclo, por lo que se ignora.
3. **Terminación.** El algoritmo termina al aceptar exactamente $n - 1$ aristas (no hace falta revisar las restantes) o al agotar la lista de aristas.

4.2. Detección y evitación de ciclos

Agregar una arista (u, v) forma un ciclo si y solo si u y v ya están en la misma componente. Para responder esta pregunta de forma eficiente se usa la estructura **Union-Find** (conjuntos disjuntos), con tres operaciones:

- **MAKE-SET(u):** crea un conjunto que contiene solo a u . Se invoca una vez por cada vértice.
- **FIND-SET(u):** devuelve el representante del conjunto que contiene a u . La arista (u, v) forma ciclo si $\text{FIND-SET}(u) = \text{FIND-SET}(v)$.
- **UNION (u, v):** fusiona los conjuntos de u y v en uno solo.

Se implementa como un bosque de árboles con dos heurísticas:

- **Unión por rango (o por tamaño):** en cada UNION, la raíz del árbol más pequeño (o de menor rango) se enlaza bajo la raíz del otro. Así la altura de los árboles se mantiene en $O(\log n)$.
- **Compresión de caminos:** durante FIND-SET(u), todos los nodos del camino recorrido pasan a apuntar directamente a la raíz, lo que acorta las búsquedas posteriores.

Con ambas heurísticas, el costo amortizado de cada operación es $O(\alpha(n))$, donde α es la inversa de la función de Ackermann.

4.3. Análisis de complejidad computacional

4.3.1. Qué representa la complejidad

Que Kruskal sea $O(m \log n)$ significa que, si se duplica el número de aristas, el tiempo de ejecución crece aproximadamente el doble (más un factor logarítmico). Como m puede ir desde $n - 1$ hasta $n(n - 1)/2$, esto equivale a:

- Grafo disperso ($m = O(n)$): $O(n \log n)$.
- Grafo denso ($m = \Theta(n^2)$): $O(n^2 \log n)$.

4.3.2. Complejidad temporal: $O(m \log n)$

Se asume un grafo simple (sin lazos ni aristas paralelas) y un ordenamiento por comparaciones de tipo mergesort o heapsort. El costo por etapas es:

Etapas	Operaciones	Costo
Inicialización	n MAKE-SET	$O(n)$
Ordenamiento	ordenar m aristas	$O(m \log m)$
Recorrido de aristas	$\leq 2m$ FIND-SET y $\leq n - 1$ UNION	$O((n + m) \alpha(n))$

Paso 1: cota superior de m . En un grafo simple, $m \leq \binom{n}{2} = \frac{n(n-1)}{2} < n^2$. Al aplicar logaritmo se obtiene la ecuación (1):

$$\log m < 2 \log n$$

Paso 2: cota inferior de m . Como G es conexo, $m \geq n - 1$. Para $n \geq 3$ se cumple $n - 1 \geq \sqrt{n}$, y entonces la ecuación (2) es:

$$\log m \geq \log(n - 1) \geq 1/2 \log n$$

De (1) y (2) se concluye que $\log m = \Theta(\log n)$ y, por lo tanto, $O(m \log m) = O(m \log n)$. Los casos $n \leq 2$ son triviales.

Paso 3: costo de Union-Find. El recorrido ejecuta a lo sumo $2m$ FIND-SET y $n - 1$ UNION, además de los n MAKE-SET. Con unión por rango y compresión de caminos, el teorema de Tarjan da un total de $O((n + m) \alpha(n))$. Como $m \geq n - 1$, se tiene $n + m \leq 2m + 1$, de modo que el costo es $O(m \alpha(n))$. En la práctica $\alpha(n) \leq 4$ para cualquier $n < 2^{65536}$, es decir, se comporta como una constante.

Paso 4: suma de las etapas. El tiempo total es la ecuación (3):

$$T(n, m) = O(n) + O(m \log m) + O(m \alpha(n))$$

Como $n \leq m + 1$ y $\alpha(n) = O(\log n)$, los términos $O(n)$ y $O(m \alpha(n))$ quedan absorbidos por el ordenamiento:

$$T(n, m) = O(m \log m) = O(m \log n)$$

Es decir, el costo está dominado por el ordenamiento inicial y Union-Find no altera el orden de crecimiento.

4.3.3. Complejidad espacial: $O(n + m)$

Estructura	Espacio
Lista de aristas (u, v, w)	$O(m)$
Union-Find (arreglos de padres y rangos)	$O(n)$
MST resultante ($n - 1$ aristas)	$O(n)$
Auxiliar del ordenamiento	$O(m)$ con mergesort; $O(1)$ con heapsort

La suma es la ecuación (4):

$$S(n, m) = O(m) + O(n) + O(n) + O(m) = O(n + m)$$

Como G es conexo, $m \geq n - 1$, por lo que también puede escribirse $O(m)$.

5. ALGORITMO DE PRIM

El algoritmo de Prim es un método voraz (*greedy*) que construye un Árbol de Expansión Mínima (MST) de un grafo $G = (V, E)$ conexo, no dirigido y con un peso $w(e)$ en cada arista. En toda la sección, $n = |V|$ es el número de vértices y $m = |E|$ el número de aristas. Prim hace crecer un único árbol a partir de un vértice inicial, incorporando en cada paso el vértice que se puede conectar al árbol con el menor costo.

5.1. Funcionamiento

- Inicialización.** Se elige un vértice inicial s . Se establece $S = \emptyset$, $clave[s] = 0$ y $clave[v] = \infty$ para todo $v \neq s$. Además, $padre[v] = NIL$.
- Selección voraz.** Se elige el vértice $u \notin S$ con menor valor de $clave[u]$ y se incorpora a S . Si $u \neq s$, la arista $(padre[u], u)$ se incorpora al MST.
- Actualización de claves.** Para cada vecino $v \notin S$ de u , si $w(u, v) < clave[v]$, se actualiza $clave[v] = w(u, v)$ y $padre[v] = u$.

4. **Terminación.** El procedimiento continúa hasta que $S = V$.

5.2. Análisis de complejidad computacional

5.2.1. Qué representa la complejidad

En Prim, esta complejidad depende de la estructura de datos usada para representar el grafo y para elegir el vértice de menor clave. Por eso se analizan varias implementaciones. Todas ejecutan las mismas operaciones básicas, y solo cambia el costo de cada una:

- n operaciones EXTRACT-MIN (una por vértice).
- A lo sumo $2m$ inspecciones de aristas, porque cada arista aparece en las listas de sus dos extremos.
- A lo sumo m operaciones DECREASE-KEY, porque una arista solo puede actualizar una clave la primera vez que uno de sus extremos sale de Q .

Con T_{ext} y T_{dec} como el costo de cada operación, el tiempo total es la ecuación (1):

$$T(n, m) = O(n) + n \cdot T_{ext} + O(m) + m \cdot T_{dec}$$

5.2.2. Complejidad temporal

a) Matriz de adyacencia y arreglo sin ordenar ($O(n^2)$): La cola de prioridad es un arreglo *clave*[] de n posiciones. Entonces:

- EXTRACT-MIN recorre el arreglo completo: $T_{ext} = O(n)$.
- Tras extraer u , se recorre su fila de la matriz (n entradas) y cada actualización es un acceso directo: $T_{dec} = O(1)$. Esto cuesta $O(n)$ por iteración.

Reemplazando en la ecuación (1), el total es la ecuación (2):

$$T(n) = O(n) + n \cdot O(n) + n \cdot O(n) = O(n^2)$$

Este tiempo no depende de m , por lo que es adecuado para grafos densos.

b) Lista de adyacencia y min-heap binario ($O(m \log n)$): El heap se construye en $O(n)$. Cada EXTRACT-MIN y cada DECREASE-KEY cuestan $O(\log n)$, es decir, $T_{ext} = T_{dec} = O(\log n)$ (para localizar el vértice dentro del heap se mantiene un arreglo auxiliar de posiciones). Recorrer todas las listas de adyacencia cuesta $O(m)$, porque la suma de los grados es $2m$. La ecuación (3) es:

$$T(n, m) = O(n) + O(n \log n) + O(m) + O(m \log n) = O((n + m) \log n)$$

Como G es conexo, $m \geq n - 1$, de modo que $n \leq m + 1 \leq 2m$ para $n \geq 2$. Entonces $n \log n \leq 2m \log n$ y la Ecuación (3) se reduce a $O(m \log n)$.

c) Lista de adyacencia y heap de Fibonacci ($O(m + n \log n)$): En un heap de Fibonacci, EXTRACT-MIN cuesta $O(\log n)$ amortizado y DECREASE-KEY cuesta $O(1)$ amortizado. La ecuación (4) da:

$$T(n, m) = O(n) + n \cdot O(\log n) + O(m) + m \cdot O(1) = O(m + n \log n)$$

Es asintóticamente mejor que la versión con heap binario en grafos densos, pero sus constantes ocultas son altas, por lo que en la práctica suele preferirse el heap binario.

d) Versión perezosa (lazy) con heap binario ($O(m \log n)$): En lugar de actualizar claves, se insertan en el heap las aristas (u, v) con $v \notin S$, y al extraer una arista se ignora si su extremo ya está en S . Cada arista se inserta a lo sumo una vez, así que hay a lo sumo m inserciones y m extracciones, cada una de costo $O(\log m)$. Como $m < n^2$ implica $\log m < 2 \log n$, la ecuación (5) es:

$$T(n, m) = O(m \log m) = O(m \log n)$$

¿Cuál conviene? El arreglo con matriz ($O(n^2)$) supera al heap binario ($O(m \log n)$) cuando $m \log n > n^2$, es decir, en grafos densos. En grafos dispersos ($m = O(n)$) el heap binario da $O(n \log n)$, mucho mejor que $O(n^2)$.

5.2.3. Complejidad espacial

Se distingue el espacio del grafo de entrada del espacio auxiliar que usa el algoritmo:

Implementación	Grafo	Auxiliar	Total
Matriz y arreglo	$O(n^2)$	<i>clave, padre</i> : $O(n)$	$O(n^2)$
Lista y heap binario (ansiosa)	$O(n + m)$	heap $\leq n$ elementos, posiciones, <i>clave, padre</i> : $O(n)$	$O(n + m)$
Lista y heap binario (perezosa)	$O(n + m)$	heap $\leq m$ aristas, <i>padre</i> : $O(m)$	$O(n + m)$

En la versión ansiosa (*eager*) el heap contiene a lo sumo un elemento por vértice no visitado, por lo que el espacio auxiliar es $O(n)$. En la perezosa, el heap puede conservar aristas ya inelegibles hasta extraerlas, y el auxiliar crece a $O(m)$.

6. COMPARACIÓN ENTRE KRUSKAL Y PRIM

Al abordar el problema del Árbol de Expansión Mínima (MST), tanto Kruskal como Prim utilizan una estrategia voraz. Ambos seleccionan en cada etapa una arista segura, cuya incorporación puede justificarse mediante la propiedad del corte. Sin embargo, difieren en la forma en que construyen el árbol, las estructuras de datos utilizadas y su comportamiento según la representación y densidad del grafo.

6.1. Tabla comparativa entre los algoritmos de Kruskal y Prim

Aspecto	Kruskal	Prim
Estrategia	Selecciona globalmente las aristas de menor peso y las incorpora si no forman ciclos.	Construye un único árbol desde un vértice inicial, añadiendo en cada paso la arista de menor peso que conecta el árbol con un vértice externo.
Estructura de datos principal	Lista de aristas ordenada y estructura Union-Find (conjuntos disjuntos).	Matriz de adyacencia con arreglos auxiliares, o lista de adyacencia con cola de prioridad (heap).
Complejidad temporal	$O(E \log E)$, equivalente a $O(E \log V)$ en un grafo simple conexo.	$O(V^2)$ con matriz de adyacencia; $O(E \log V)$ con lista de adyacencia y heap binario.
Grafos dispersos	Muy conveniente, especialmente cuando el número de aristas E es relativamente pequeño.	También es eficiente con lista de adyacencia y heap binario.
Grafos densos	El ordenamiento global puede involucrar una gran cantidad de aristas.	La implementación con matriz de adyacencia y $O(V^2)$ suele ser conveniente.
Ventaja principal	Trabaja directamente con las aristas y detecta ciclos eficientemente mediante Union-Find.	No requiere ordenar globalmente todas las aristas y puede aprovechar eficientemente una representación por matriz en grafos densos.
Desventaja principal	Necesita ordenar previamente las aristas; además requiere Union-Find para evitar ciclos.	Con heap, su implementación es más elaborada; con matriz, requiere $O(V^2)$ de memoria para almacenar el grafo.
Facilidad de implementación en C	Relativamente directa si se dispone de una lista de aristas; requiere implementar correctamente Union-Find y el ordenamiento.	La versión con matriz es sencilla; la versión con heap requiere una cola de prioridad y estructuras auxiliares.
Grafo desconexo	Produce naturalmente un bosque de expansión mínimo, uno para cada componente conexa.	Una ejecución desde un vértice solo cubre su componente conexa; para obtener un bosque mínimo debe reiniciarse en las componentes restantes.

Aspecto	Kruskal	Prim
Cuándo conviene utilizarlo	Cuando el grafo es disperso, la entrada está dada como lista de aristas o se desea obtener directamente un bosque de expansión mínimo.	Cuando el grafo es denso y se representa mediante matriz, o cuando se dispone de listas de adyacencia y una cola de prioridad eficiente.

6.2. Consideraciones para su implementación en Lenguaje C

En la implementación de Kruskal, la principal consideración es la detección eficiente de ciclos. Para ello se utiliza la estructura de conjuntos disjuntos Union-Find, implementada mediante arreglos de padres y rangos, junto con compresión de caminos y unión por rango. Las aristas se almacenan en un arreglo y se ordenan de forma ascendente antes de iniciar la selección.

En Prim, la implementación depende de la representación del grafo. En este trabajo se utiliza una matriz de adyacencia junto con arreglos auxiliares para almacenar las claves mínimas, los vértices seleccionados y los padres de cada vértice. Esta alternativa presenta una complejidad temporal de $O(V^2)$ y resulta sencilla de implementar en C.

7. SELECCIÓN DEL ALGORITMO

Para determinar cuál de los dos algoritmos resulta más conveniente en un problema específico, la elección debe basarse en las características del grafo, la representación de los datos y las estructuras disponibles. Los principales criterios que se deben considerar son:

- 1. La densidad del grafo ($|E|$ frente a $|V|$):**
Si el grafo es disperso y el número de aristas es relativamente pequeño respecto al número de vértices, Kruskal suele resultar conveniente, especialmente cuando la entrada se proporciona como una lista de aristas. En grafos densos, Prim implementado mediante una matriz de adyacencia puede resultar conveniente, ya que presenta una complejidad temporal de $O(V^2)$ y evita realizar un ordenamiento global de las aristas.
- 2. Representación de los datos y estructura del grafo:**
Si el grafo se proporciona principalmente como una lista de aristas y presenta una cantidad relativamente pequeña de conexiones, Kruskal resulta una alternativa natural, ya que opera directamente sobre dichas aristas. En cambio, si el grafo es denso y se encuentra representado mediante una matriz de adyacencia, Prim puede resultar conveniente debido a que su implementación con matriz presenta una complejidad temporal de $O(V^2)$.

8. EVIDENCIAS DE EJECUCIÓN

Para verificar el correcto funcionamiento de ambas implementaciones, se utilizó el mismo grafo de entrada. Tanto Kruskal como Prim obtuvieron un árbol de expansión mínima de costo total 19.

```
--- ARBOL DE EXPANSION MINIMA (KRUSKAL) ---
Arista (Origen - Destino)      Peso
    2 <---> 3                   4
    0 <---> 3                   5
    0 <---> 1                   10
Costo total del MST: 19

-----
Process exited after 0.01939 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 1. Ejecución de Kruskal.

```
=== RESULTADO DEL MST (ALGORITMO DE PRIM) ===
Arista (Origen - Destino)      Peso
    0 <---> 1                   10
    3 <---> 2                   4
    0 <---> 3                   5
Costo total minimo de la red: 19

-----
Process exited after 0.01518 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2. Ejecución de Prim.

9. CONCLUSIONES

El estudio de los árboles de expansión mínima permite comprender cómo es posible conectar todos los vértices de un grafo ponderado reduciendo al mínimo el costo total de sus aristas y evitando la formación de ciclos. En este contexto, los algoritmos de Kruskal y Prim constituyen dos métodos voraces capaces de resolver el mismo problema mediante estrategias diferentes: Kruskal selecciona aristas de forma global, mientras que Prim construye progresivamente un único árbol a partir de un vértice inicial.

A partir del análisis realizado, se observa que la eficiencia de cada algoritmo depende en gran medida de la representación del grafo y de las estructuras de datos utilizadas. Kruskal presenta un costo dominado por el ordenamiento de las aristas y se beneficia del uso de Union-Find para la detección eficiente de ciclos, mientras que Prim puede implementarse con matriz de adyacencia o con listas de adyacencia y colas de prioridad, obteniendo distintas complejidades según el caso.

La comparación desarrollada muestra que no existe una única implementación conveniente para todos los grafos. Kruskal resulta especialmente natural cuando la entrada se presenta como una lista de aristas o cuando se trabaja con grafos dispersos, mientras que Prim puede ser adecuado en grafos densos representados mediante matrices de adyacencia. Asimismo, ante grafos desconectados, Kruskal genera de forma natural un bosque de expansión mínimo, mientras que Prim debe aplicarse por separado en cada componente conexa.

Finalmente, la implementación de ambos algoritmos en lenguaje C permitió relacionar el análisis teórico con su aplicación práctica. Las evidencias de ejecución muestran que ambos métodos pueden obtener un árbol de expansión mínima con el mismo costo total para un mismo grafo de entrada, confirmando el correcto funcionamiento de las implementaciones desarrolladas.

10. BIBLIOGRAFÍA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Kleinberg, J., & Tardos, É. (2006). Algorithm Design. Pearson Education.
- Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. Proceedings of the American Mathematical Society, 7(1), 48–50.
- Prim, R. C. (1957). Shortest connection networks and some generalizations. Bell System Technical Journal, 36(6), 1389–1401.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- Weiss, M. A. (1997). Data Structures and Algorithm Analysis in C (2nd ed.). Addison-Wesley